Question 7: Prepare a monitoring, logging, and DevSecOps report describing the automation, observability setup, security findings, and observations with screenshots.

Executed on Ubuntu 24.04 LTS. All commands, configurations and screenshots are committed in the GitHub repository https://github.com/theabhinav0231/devops-lab-3.

1. Objective

Build an automated, observable and secure infrastructure workflow around the Flask application: Ansible and Terraform for automation, Prometheus/Grafana for metrics, the ELK stack for centralized logging, and DevSecOps scanning plus feature flags, load and browser testing integrated into the process.

2. Tools and environment

- Ubuntu 24.04 LTS (WSL2), Docker Engine 29.9.2, Docker Compose v5.7.0
- Ansible core 2.21.5, Terraform v1.17.1 (docker provider)
- Prometheus v2.53, Grafana 11.1, Elasticsearch/Logstash/Kibana 8.14.1
- Trivy 0.76.1, pip-audit, bandit, Selenium 4 + Chrome/ChromeDriver 157, apache bench

3. Architecture

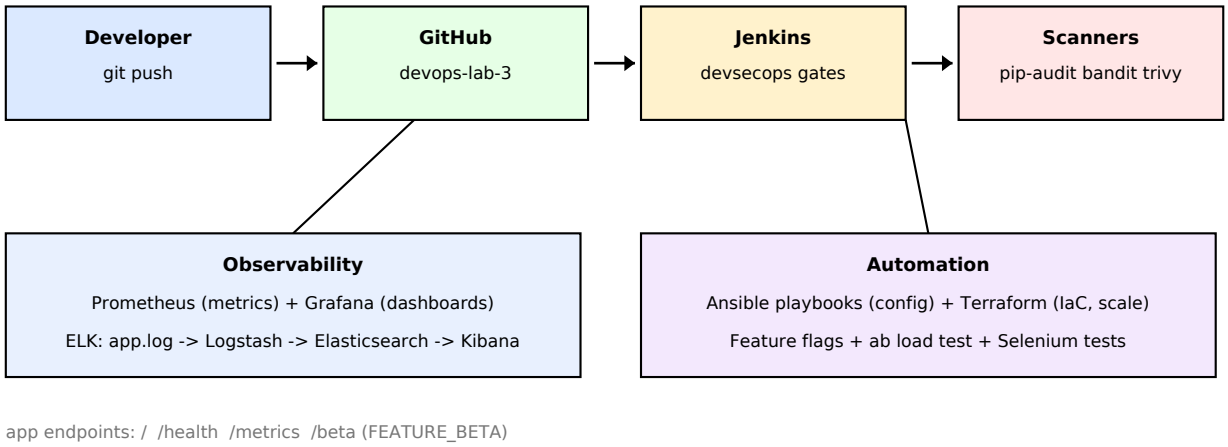


Fig: Lab 3 monitoring, logging and DevSecOps architecture

4. Automation results

Tool	What it automated	Evidence
Ansible	nginx install + app page on the managed node	playbook recap ok=6 changed=0 (2nd run)
Terraform	2 -> 4 nginx containers via one variable	apply outputs + terraform state list
Jenkins	security gates on every change (poll SCM)	builds 1-3 blocked, build 4 passed

5. Observability setup

- Metrics: prometheus_client counter on every response; 5s scrape; Grafana dashboard with req/s, totals and target uptime, provisioned from files.

- Logs: Flask file logging -> Logstash grok parse -> Elasticsearch index lab3-app-logs (765+ documents) -> Kibana Discover with structured level/message fields.
- Both observability stacks are single docker-compose files - reproducible on any machine.

6. Security findings

Scanner	Finding	Resolution
pip-audit	flask 3.1.0: PYSEC-2026-1377/2151; pytest 8.3.4: PYSEC-2026-1845	bumped to flask 3.1.3 / pytest 9.0.3 - audit clean
bandit	B108 hardcoded /tmp dir; B104 bind-all-interfaces	B108 fixed in code; B104 skipped with documented container justification
trivy	real HIGH/CRITICAL base-image CVEs (util-linux, perl-base) in the lab2 image	report-only; base image refresh queued as follow-up

7. Testing results

- Feature flag: same image, /beta 404 (off) vs 200 (on) - toggle without redeploy.
- Load test (ab, 2000 req, c=50): 323.74 req/s, 0 failed, p95 211 ms, p99 267 ms.
- Selenium (headless Chrome 157): /health asserted, enabled beta flag asserted, screenshot captured - 3/3 PASS.

8. Evidence

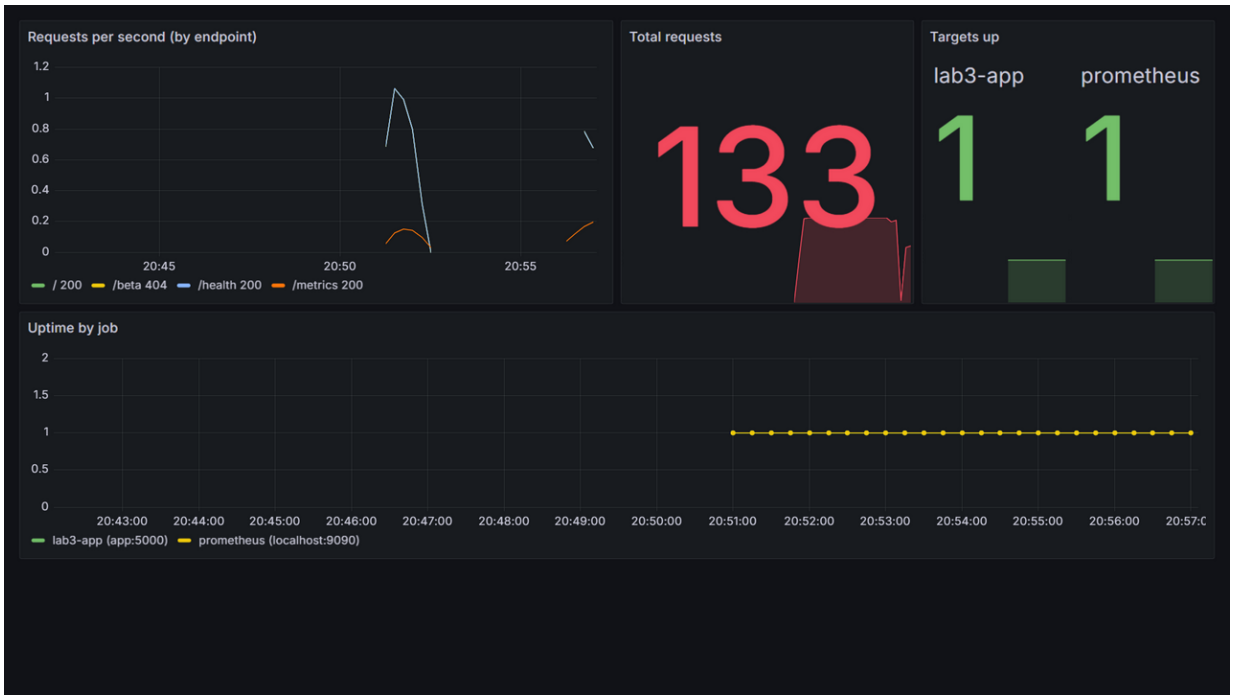


Fig: Grafana dashboard with live metrics

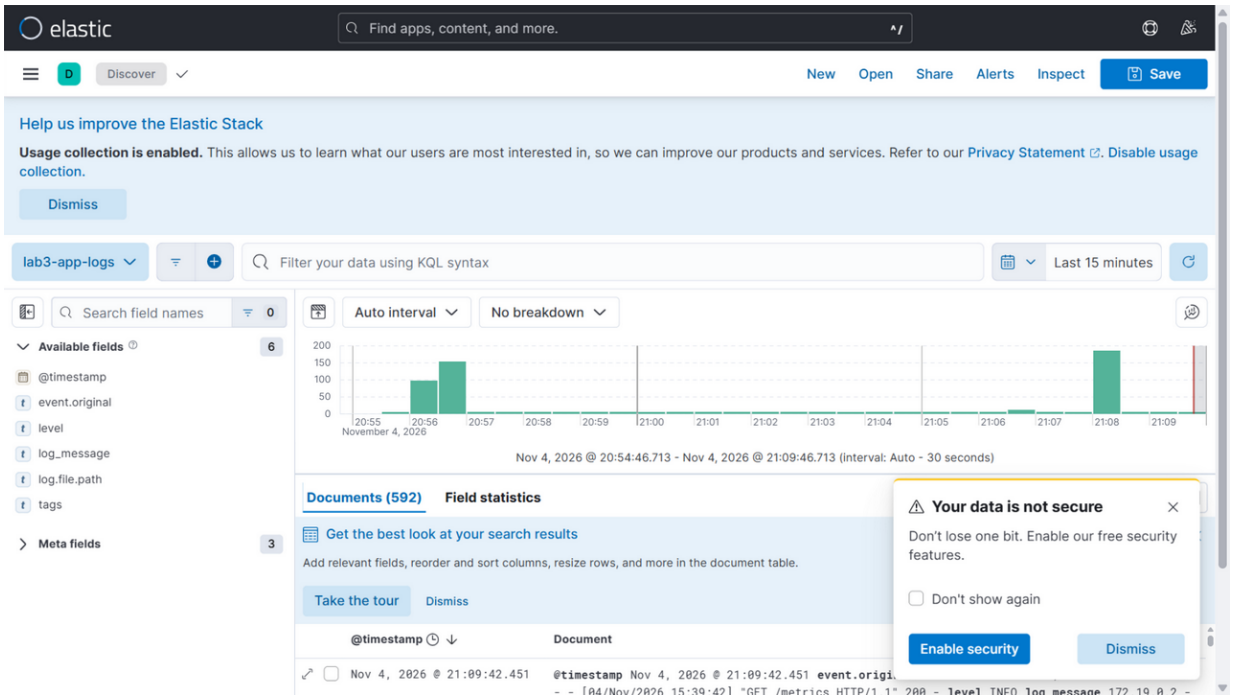


Fig: Kibana Discover with parsed logs

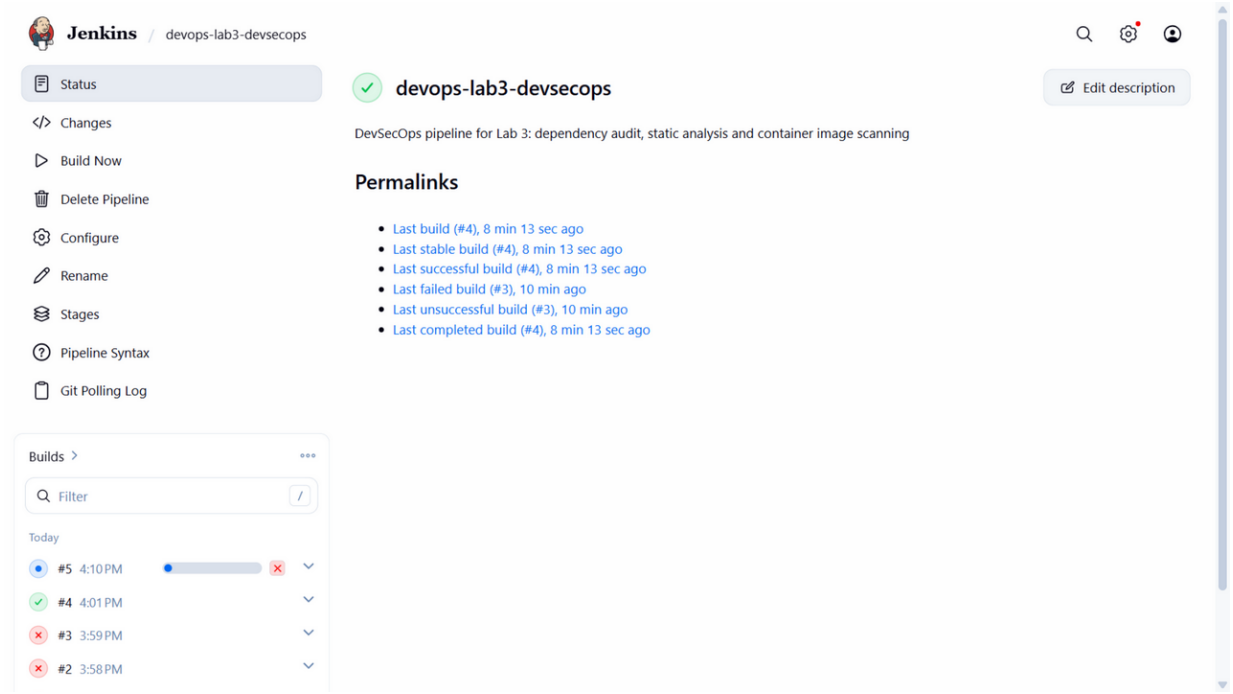


Fig: DevSecOps pipeline history

Pretty-print ☐

{"enabled":true,"feature":"beta","message":"Welcome to the beta feature"}

Fig: Selenium browser screenshot

9. Observations

- Ansible's docker connection plugin removed all SSH setup pain for the managed node.
- Terraform plan is the safety net: the scale change was reviewed as '2 to add, 0 to change' before applying.
- Provisioned Grafana (datasource + dashboard as files) means the dashboard exists on first boot - no clicks.
- Grok parsing is what makes logs searchable; raw lines alone would make Kibana useless.
- The security gate caught real advisories (flask 3.1.0) - proof that dependency scanning belongs in CI, not in an occasional manual audit.
- Bandit exceptions must be documented; B104 (bind all interfaces) is genuinely required inside containers.
- ELK needs host tuning in WSL2 (vm.max_map_count) and is memory hungry - sized down to 512m heap for the lab.

10. Conclusion

The workflow is now automated (Ansible, Terraform), observable (Prometheus/Grafana metrics, ELK centralized logs) and secure (pip-audit, bandit and trivy gates in the CI/CD pipeline, with two real blocks demonstrated). Feature flags, load testing and Selenium browser testing complete the modern DevOps practice loop. All evidence is committed in github.com/theabhinav0231/devops-lab-3.